Information Reconciliation Algorithms for QKD

Attempt Implementation of an Unified Method of Evaluating Error Correction Protocols

Master in Quantum Computing - University POLITEHNICA of Bucharest

May 27th, 2026

Alexandra Chiriță **Sergiu Manoilă**

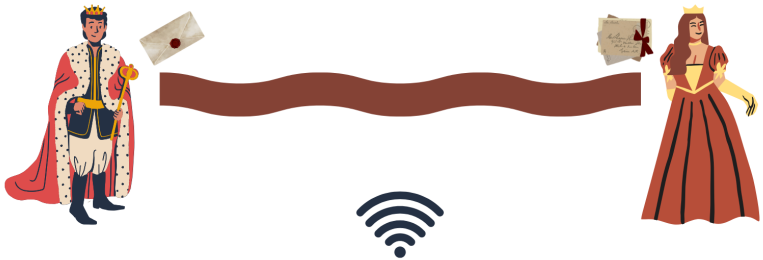
Table of Contents

Harap Alb (Prince)

Ileana Cosânzeana (Princess)

Harap Alb (Prince)

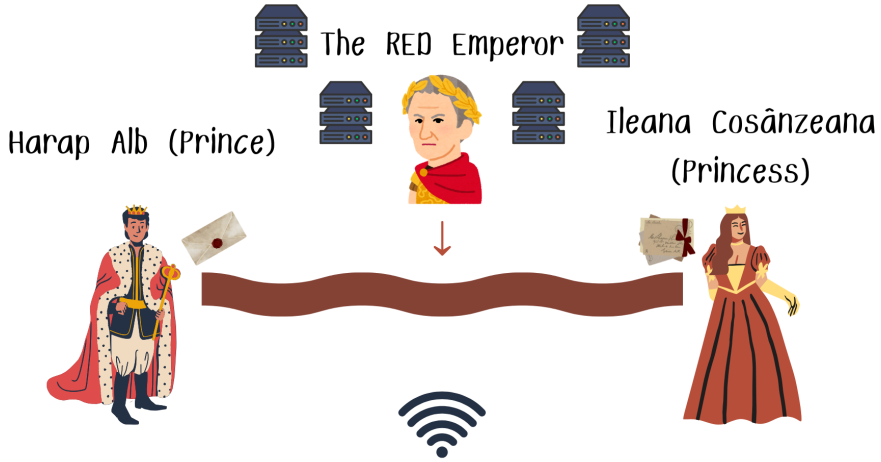
Ileana Cosânzeana (Princess)



The RED Emperor

Harap Alb (Prince)

Ileana Cosânzeana (Princess)


The RED Emperor
Harap Alb (Prince)
Ileana Cosânzeana
(Princess)

Hence our Motivation

Why work on the faculty's QKD network

- Our first application on a QKD network
- A chance to do something useful. Conduct a real experiment on state-of-the-art protocols.
- One of the papers we picked conducted their experiment on a phyical QKD system [Mueller et al., 2025]

Problems

- We need to exchange certificate keys between the VM and the node
- Limited availability of the network administrator

So Back to Simulators

SeQUeNCe [Wu et al., 2021]

Open-source, discrete-event simulator for quantum networks from Argonne National Lab & the University of Chicago.

What we use it for?

- Run a single-link BB84 session between two nodes (the Prince & the Princess).
- Produce realistic pairs of bits that Harap Alb and the Princess hold after the quantum exchange via a noisy channel.
- Sweep channel parameters (loss, dark counts, distance) to drive the error rate from near-zero up to highly noisy regimes, covering the conditions the three papers evaluate their algorithms on.

Why SeQUeNCe?

Why we picked this simulator instead of other?

- Python library
- Simulate the whole network, not just point-to-point links
- **Flexible input:** JSON files
- Supports billions of network events
- *Not the only option:* NetSquid, QuISP, SimulaQron, QuNetSim are notable alternatives.[a]

[a] netsquid.org · quisp · simulaqron.org · QuNetSim

Information Reconciliation — Two Families

Goal

Harap Alb and the Princess correct mismatches in their raw keys using *only classical communication*, while revealing as little key material as possible.

Interactive (syndrome-free)

- Harap Alb and the Princess exchange *parities* of blocks in multiple rounds.
- Errors are located by binary search.

$\Rightarrow$ **Cascade** [Brassard and Salvail, 1994]

Coding-based (one-shot)

- The Princess sends a *syndrome* derived from an error-correcting code.
- Harap Alb decodes locally using belief propagation.

$\Rightarrow$ **Low-Density Parity-Check codes** (LDPC) [Gallager, 1962]

Both achieve the same goal; they differ in round complexity, throughput, and leakage.

Information Reconciliation — Algorithm Landscape

Goal

Harap Alb and the Princess correct mismatches using *only classical communication*, leaking as little key material as possible to an eavesdropper.

Algorithm	Reference	Type	Key idea
Cascade	[Brassard and Salvail, 1994]	Interactive	Multi-pass parity comparison + binary search
Winnow	[Buttler et al., 2003]	Interactive	Hamming-code syndromes; fewer round-trips
Turbo codes	[Berrou et al., 1993]	Coding-based	Two convolutional codes, iterative decoding
LDPC	[Martinez-Mateo et al., 2015]	Coding-based	Sparse parity-check matrix + belief propagation
Polar codes	[Jouguet and Kunz-Jacques, 2014]	Coding-based	Capacity-achieving; successive cancellation

Trend: interactive multi-round protocols → one-shot codes near the Shannon limit.

Cascade: Interactive Error Correction

Core idea [Brassard and Salvail, 1994]

Divide the key into blocks, compare parities, and *binary-search* for each detected error — repeat with larger blocks to catch errors exposed by bit flips.

Algorithm sketch

1. Split n bits into blocks of size k_1.
2. Harap Alb announces block parities; The Princess compares.
3. Mismatch $\Rightarrow$ `BINARY` search to flip the error.
4. Bit flip can break earlier parities $\Rightarrow$ back-track.
5. Repeat with $k_2, k_3, \ldots$ until no mismatches.

Properties

- No prior QBER estimate — works "blindly".
- Leakage $\approx 1.16\, h(e)$ bits/bit; optimized variants reach ≈ 1.06 [Martinez-Mateo et al., 2015].
- Many rounds: $O(\log n)$ trips per error.
- Sequential by design — hard to parallelize.

LDPC: Coding-Based Reconciliation

Core idea [Elkouss et al., 2009]

Harap Alb encodes her key with a *linear* LDPC code and sends the **syndrome** $\mathbf{s} = H\mathbf{x}$ to The Princess. The Princess runs *belief propagation* to recover $\mathbf{x}$ from $\mathbf{y}$ and $\mathbf{s}$.

Algorithm sketch

1. Pick rate-R parity-check matrix H matched to QBER.
2. Send syndrome $\mathbf{s} = H\mathbf{x}$ ($n(1-R)$ bits).
3. The Princess initialises BP with channel LLRs; iterates to convergence.
4. **Blind protocol:** on BP failure, reveal extra bits and retry [Martinez-Mateo et al., 2012, Mueller et al., 2025].

Properties

- Single round — one syndrome message.
- Leakage approaches $h(e)$ (Shannon limit).
- Highly parallel — GPU/FPGA friendly.
- Needs QBER estimate (or blind overhead).
- AEC/SEC variants for asymmetric loads [Borisov et al., 2023].

LDPC vs. Cascade: What Is Improved

	Cascade	**LDPC**
Rounds	Many ($O(\log n)$ per error)	**1** (syndrome + hash)
Latency	High — each round needs a reply	**Low** — fire-and-forget
Throughput	Sequential, CPU-bound	**Parallelisable** (GPU/FPGA)
Leakage	≈ 1.06–$1.16\, h(e)$	$\to h(e)$ (near-Shannon)
QBER knowledge	Not required	Required (or blind overhead)
Block size	Moderate (10^3–10^4)	Scales to 10^6+ bits

Bottom line

- LDPC eliminates the interactive back-and-forth of Cascade, enabling **higher key rates** and **lower latency**.
- **Cost:** we need a good QBER estimate or a blind-rate protocol [Mueller et al., 2025].

3 papers. One story

Cascade: Original & Optimized Variants (opt.7–8) [Martinez-Mateo et al., 2015]

Systematic study on optimization of the Bassard's Cascade [Brassard and Salvail, 1994].
Baseline for the current state-of-the-art (SOTA) [Pacher et al., 2015].

Cascade vs. LDPC with the Blind Protocol [Mueller et al., 2025]

Benchmark of SOTA Cascade [Pacher et al., 2015] against LDPC + Martinez-Mateo [Martinez-Mateo et al., 2012] blind disclosure on industrial QKD system.
Inspiration for our analysis report.

Asymmetric Adaptive LDPC with Rate Pool [Borisov et al., 2023]

Per-frame rate selection from a pre-built LDPC code pool with a disclosure rescue loop, evaluated on industrial QKD traces at target $f \approx 1.15$.

- We need a unified set of metrics to compare them. How do we measure them?

Paper 1: Tuning Cascade to Its Limit [Martinez-Mateo et al., 2015]

The question

Cascade has been around since 1994. Nobody had systematically asked: *what are the best possible settings?* This paper tests 8 variants and finds the answer.

What they tuned

- How many bits to put in each block per pass
- How long each frame should be
- Whether to shuffle bits between passes

Winner: 16 384-bit frames, 14 passes, block sizes that grow with each pass.

✓ We use these exact settings in our implementation.

Result

- Wastes **6% more key than the theoretical minimum** — down from 16% with the original 1994 version
- The size of the first block matters most; everything else is secondary

The ceiling

This is about as good as Cascade can get. It still needs many back-and-forth messages — one per error found.

Paper 1: Three Tricks That Save Key Bits [Martinez-Mateo et al., 2015]

1. Don't redo work when backtracking

When Cascade goes back to fix errors it created in earlier passes, it reuses sub-blocks it already split — instead of starting the binary search from scratch. Fewer messages sent.

2. Skip shuffling on the first pass

Errors are randomly scattered at the start, so the natural bit order works just as well as a random shuffle. One less computation.

3. The last block's parity is free

The XOR of all the block parities in a pass always equals the total parity of the whole frame — which both The Prince and The Princess already know. So the last block never needs to be announced. **Saves 1 bit per pass = 14 bits per frame.**

Together

These three tricks drop leakage from 16% above the Shannon limit to just 6% above it.

Cascade is now as efficient as it can practically get. Does LDPC beat it? → Paper 2

Paper 2: LDPC vs. Cascade on a Real QKD Link [Mueller et al., 2025]

Builds on Paper 1 — uses the best Cascade as the baseline

Runs both algorithms on an actual physical QKD machine (not a simulation) to see which wastes less key and runs faster.

How they measured it

Effective efficiency f_{eff}: how much extra key was wasted compared to the theoretical minimum.

- $f_{\text{eff}} = 1.0 \rightarrow$ perfect, no waste at all
- $f_{\text{eff}} = 1.06 \rightarrow 6\%$ wasted (best Cascade)
- Lower is always better

Results

- LDPC is always **faster** — it only needs one message instead of dozens
- At low error rates ($<3\%$): both waste about the same amount of key
- At high error rates ($>5\%$): LDPC wastes **10–20% less**

Catch: LDPC needs to know the error rate upfront. $\rightarrow$ next slide

Paper 2: What If You Don't Know the Error Rate? [Mueller et al., 2025]

The problem

LDPC is like picking a box before you know how much stuff you're packing. Pick too small a box — it overflows, decoding fails, you have to try again.

The blind protocol — try, then adjust

1. Make a best guess at the error rate; send the syndrome.
2. If decoding fails, reveal a small batch of extra bits to give the decoder more information.
3. Try again. If it still fails, reveal another batch.
4. Stop when decoding succeeds or you run out of budget.

In practice this usually succeeds in 1–2 rounds.

How success is confirmed

Once decoding converges, a hash check verifies the result. A quick pre-check first filters out obvious failures without the expensive hash.

Every retry costs one extra message and reveals extra bits. If the error rate changes a lot between frames, retries add up.

Paper 3: LDPC That Learns From Experience [Borisov et al., 2023]

Builds on Paper 2 — fixes the guessing problem

Instead of guessing the error rate fresh each frame, keep a running average of recent frames. Guess right on the first try most of the time.

How the tracker works

- Keeps a **weighted average** of the last 6 frames — recent ones count more
- If decoding fails: assume the error rate was high (penalty value), stay conservative
- **Sudden spike detector:** if the error rate jumps far outside the norm, override the average immediately and use the raw current value

AEC vs. SEC

- **AEC** (Asymmetric):
  The Prince sends one syndrome message to The Princess. Simple, one-directional, low overhead.
- **SEC** (Symmetric):
  Both sides send parities to each other. Used when neither party's key is the clear reference; costs two messages instead of one.

Paper 3: Picking the Right Code & Revealing Bits [Borisov et al., 2023]

Choosing the code rate

From a pre-built pool of codes at different rates, pick the *highest rate* (least redundancy) that still has enough slack to handle the estimated error rate. Higher rate = less leakage.

What "slack" means: the code has reserved positions in the codeword that can be used to absorb more errors if needed — like packing extra bubble wrap in case the box is overfull.

When decoding fails — what to reveal

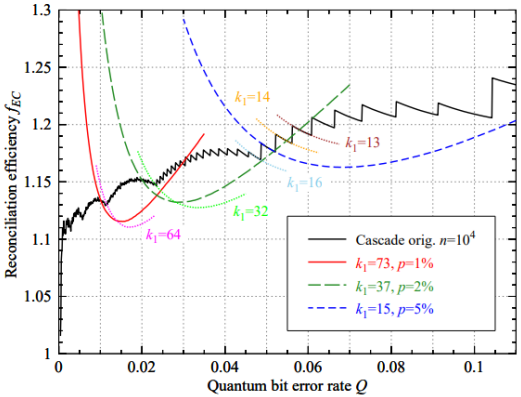
Reveal just enough extra bits to push the decoder over the threshold. Priority order:

1. Reserved positions first — revealing one frees up a syndrome constraint (double value)
2. Then random payload bits — each one directly helps the decoder

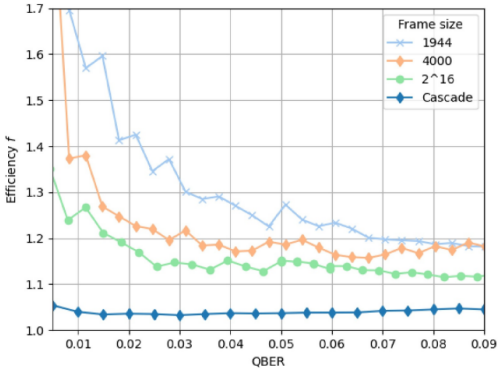
The decoder itself

Uses a simplified version of belief propagation (Min-Sum, scaling 0.875) — slightly less precise but faster and easier to run on hardware.

How to compare these papers?



Cascade efficiency f_{EC} vs QBER for several initial block sizes k_1 [Martinez-Mateo et al., 2015].



LDPC efficiency f vs QBER for frame sizes 1944, 4000, 2^{16}, compared with Cascade [Mueller et al., 2025].

Unified Evaluation Metrics

Symbol	Code	Description	Paper
Q	`q`	QBER of the sifted key; horizontal axis on every plot (channel-level error rate on BB84 sifted bits).	[1,2,3]
f	`f`	Reconciliation efficiency $f = L/(n\,h_2(Q))$; f=1 is the Shannon limit. Lower is better.	[1]
FER	`fer`	Frame error rate; fraction of frames with $\hat{x} \neq x_A$ after IR, with Wilson 95% CI.	[2,3]
R_{sec}	`r_sec`	Asymptotic secret-key rate per sifted bit: $R_{\text{sec}} = 1 - f\,h_2(Q) - h_2(Q)$.	[1]
M	`messages`	Round-trip messages per frame (Cascade passes; LDPC = 1 + disclosure rounds).	[1,2]
T_{it}	`iterations`	BP iterations per frame until syndrome match or max budget (LDPC algorithms only).	[2,3]

Metrics taken from the source papers: [1] Martínez-Mateo et al., 2015 [Martinez-Mateo et al., 2015]; [2] Müller et al., 2025 [Mueller et al., 2025]; [3] Borisov et al., 2023 [Borisov et al., 2023].

Sweep Parameters — This Work vs. Source Papers

Parameter	This work	Mart.-Mateo 2015 [1]	Müller 2025 [2]	Borisov 2023 [3]
Frame length N	1024	10^4 (and 2^{14} opt.)	1944, 4000, 2^{16}	32,000
Block / subblock	N (per frame)	$n = 10^4$	$n = N$	$\ell_{\text{sub}} = 27{,}200$
Frames per Q	200	$> 10^6$ (for FER)	$\sim 10^3$ (live: $\sim$ 27h)	4.5×10^4
QBER range	[0.005, 0.10]	[0, 0.11]	[0.005, 0.09]	[0.005, 0.105]
# QBER points	20	~ 50	~ 20	21
LDPC code pool R	`Elkouss` $\{0.5, \ldots, 0.9\}$	—	$\{R_0, \ldots, R_9\}$ Elkouss	$\{0.5, 0.55, \ldots, 0.9\}$
Rate-adapt α	0.15	—	step size $\alpha = 1$	0.15
Cascade k_1	$2^{\lceil \log_2 1/Q \rceil}$ (opt. 7)	$2^{\lceil \log_2 1/Q \rceil}$ (opt. 7/8)	opt. 7/8 (Pacher)	—
Cascade passes	14	14 (opt. 7/8)	14	—
BP iterations	≤ 50	—	50 (SPA)	var-scaled min-sum
EMA factor γ	0.33	—	—	0.33
Seed efficiency f_{start}	1.15	—	—	1.15
Burst threshold σ_m	3σ	—	—	3σ
Channel model	BSC (numpy)	BSC (BB84 sifted)	BSC (depolarising)	BSC (decoy-BB84)
Frame source	BSC + SeQUeNCe	simulated	live QKD + simul.	QRate device + simul.

Sources: [1] Martínez-Mateo et al. 2015 [Martinez-Mateo et al., 2015]; [2] Müller et al. 2025 [Mueller et al., 2025]; [3] Borisov et al. 2023 [Borisov et al., 2023].

Source: Wikimedia Commons

Prerequisites for Metrics

Quantum Bit Error Rate (QBER)

The fraction of mismatched bits between The Prince's and The Princess's sifted keys. Sets how hard the reconciliation step has to work. [Martinez-Mateo et al., 2015]

$$Q = \frac{N_{\text{errors}}}{N_{\text{sifted}}}$$

Shannon limit

To fix the Princess's noisy bits, Harap Alb must reveal a minimum amount of information on the public channel — set by the channel noise, not by how clever the protocol is. No protocol can do better. f measures how close real protocols get to this floor: $f = 1$ is perfect, $f > 1$ means wasted bits. [Martinez-Mateo et al., 2015]

Reconciliation efficiency f

"Headline" metric from all three papers

Measures how close the protocol is to the theoretical Shannon limit. Or how expensive is the protocol in leaked bits [Martinez-Mateo et al., 2015]

$$f = \frac{\text{leak}_{IR}}{n\,H(X|Y)} = \frac{\text{leak}_{IR}}{n\,h(q)}$$

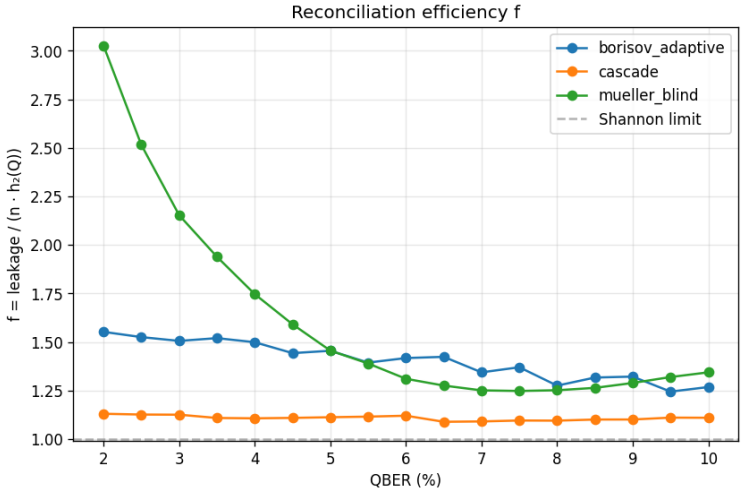
$$h(q) = -q\log_2 q - (1-q)\log_2(1-q)$$

Reconciliation efficiency f

$$f = \frac{\text{leak}_{IR}}{n\,H(X|Y)} = \frac{\text{leak}_{IR}}{n\,h(q)}, h(q) = -q\log_2 q - (1-q)\log_2(1-q)$$

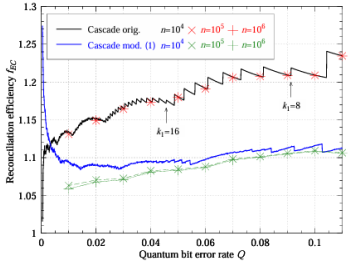
- f - reconciliation efficiency; $f = 1$ is the ideal limit, $f > 1$ in practice.
- leak_{IR} - number of bits leaked during reconciliation.
- n - frame length (number of key bits reconciled per block).
- $H(X|Y)$ - conditional entropy; the minimum information needed to reconcile.
- q - quantum bit error rate (QBER).
- $h(q)$ - binary entropy of q; equals $H(X|Y)$ for a BSC.

Reconciliation efficiency vs QBER ($N = 1024$)



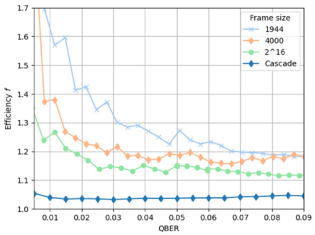
Cascade hugs Shannon; LDPC variants pay a finite-N + pool penalty.

Reference figures from the three papers



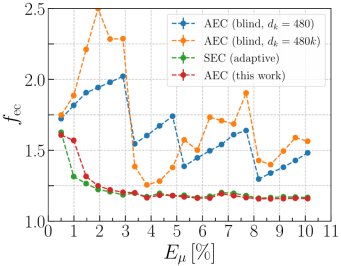
(a) Cascade efficiency f

Martinez-Mateo 2014



(b) LDPC blind efficiency f

Mueller 2025



(c) FEC performance

Borisov 2023

All three papers report reconciliation behavior vs QBER; Borisov frames it as FEC performance directly.

Effective efficiency f_{eff}

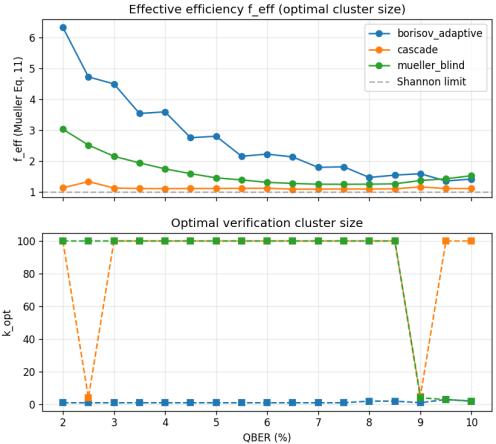
Definition

The effective efficiency f_{eff} extends f by also including the cost of error verification (EV) and the leakage from failed frames.

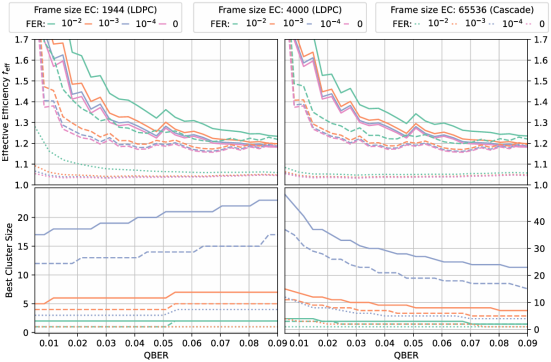
$$f_{\text{eff}} = (1 - \text{FER}_{\text{cluster}} - P_{\text{Collision}})\, f + \frac{\text{FER}_{\text{cluster}} + P_{\text{Collision}}}{h(q)} + \frac{t}{n\, h(q)}$$

- f - reconciliation efficiency when decoding succeeds.
- $\text{FER}_{\text{cluster}} = 1 - (1 - \text{FER})^k$ - failure probability of a cluster of k frames.
- $P_{\text{Collision}}$ - probability of an EV hash collision (undetected failure).
- t - number of bits used for the verification tag.
- n - frame size of error correction.
- $h(q)$ - binary entropy at QBER q.

Effective efficiency with verification cluster ($N = 1024$)



Our run, $N = 1024$. f_{eff} from Mueller Eq. 11; $k^\star$ adapts to FER.



Mueller 2025, Fig. 9. Top: f_{eff} at optimal k. Bottom: optimal k. Right column allows retry on failure.

Frame error rate

Definition

The **frame error rate (FER)** is the probability that, after reconciliation, The Prince's and The Princess's frames still differ by at least one bit (i.e the probability that reconciliation fails).

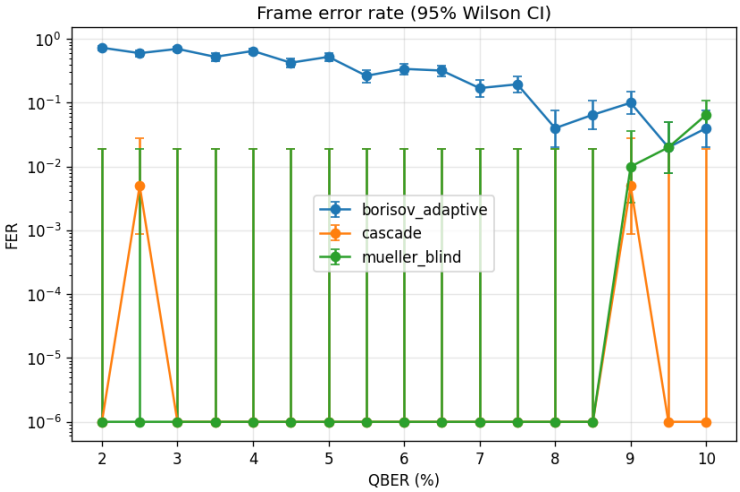
The "Catch"

Efficiency f alone is misleading: a code may look efficient but fail often. When reconciliation fails, the entire frame (n bits) is considered leaked. FER captures this trade-off. Lower f often comes at the cost of higher FER.

$$f_{\text{FER}} = (1 - \text{FER})\, f + \frac{\text{FER}}{h(q)}$$

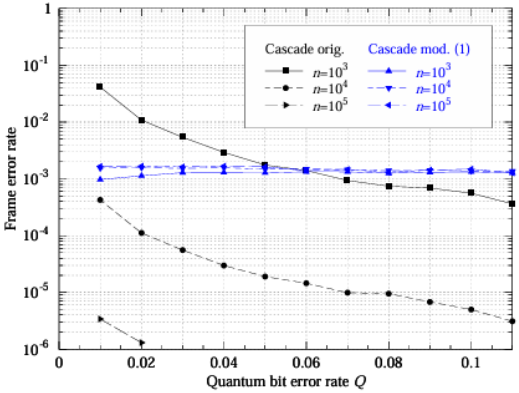
- f_{FER} - effective efficiency including failed frames.
- f - reconciliation efficiency when decoding succeeds.
- FER - frame error rate (failure probability).

Frame error rate ($N = 1024$)

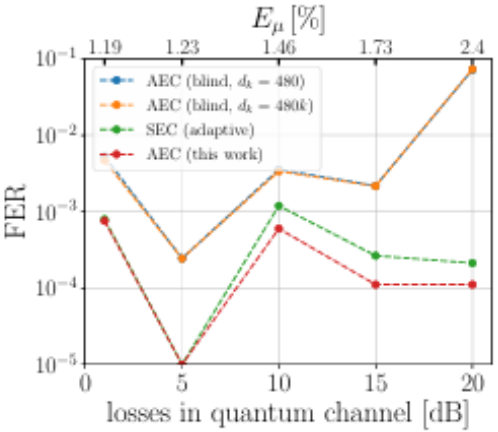


Cascade and Mueller decode reliably across the range; Borisov shows the placeholder-pool effect at low Q.

Frame error rate — paper references



Martinez-Mateo 2014. FER for Cascade across QBER; protocol iterates to convergence so FER stays near zero.



Borisov 2023. FER for asymmetric adaptive LDPC; $\sim 10^{-3}$ across industrial QBER thanks to per-rate disclosure retry.

Messages per reconciled bit

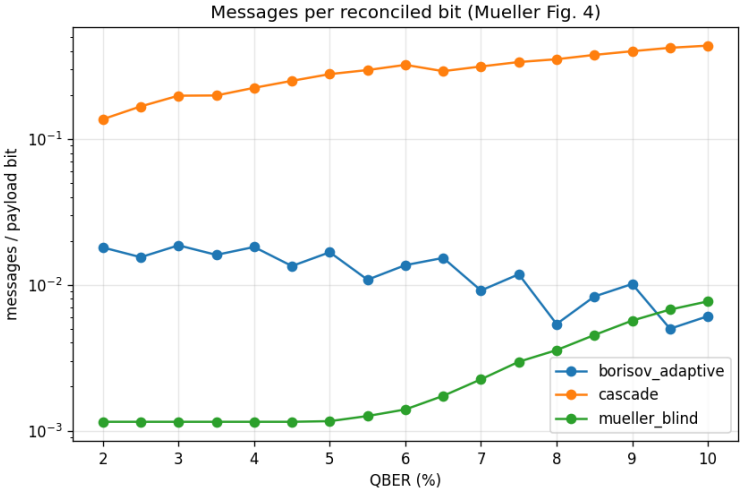
Definition

The **messages per reconciled bit** measures how many messages The Prince and The Princess exchange on the classical channel for each bit of reconciled key.

$$m_{\text{bit}} = \frac{M}{n}$$

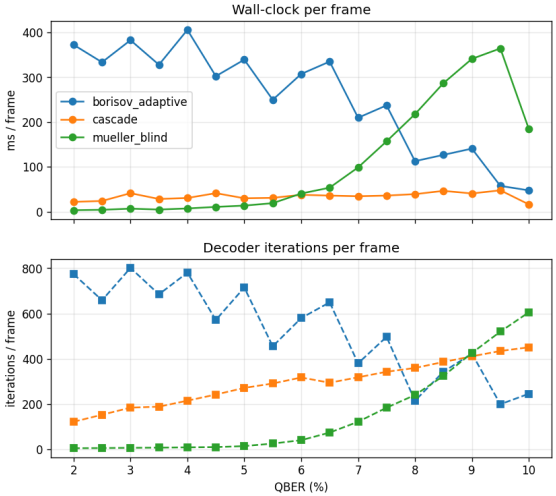
- m_{bit} - mean number of messages exchanged per reconciled bit.
- M - total number of messages exchanged between The Prince and The Princess during reconciliation of one frame.
- n - frame length (number of key bits reconciled).

Communication overhead ($N = 1024$)



Cascade pays in round-trips; LDPC variants finish in a handful of messages.

Computational cost ($N = 1024$)



Wall-clock per frame (top) and BP / pass iterations (bottom).

Robustness to QBER mismatch

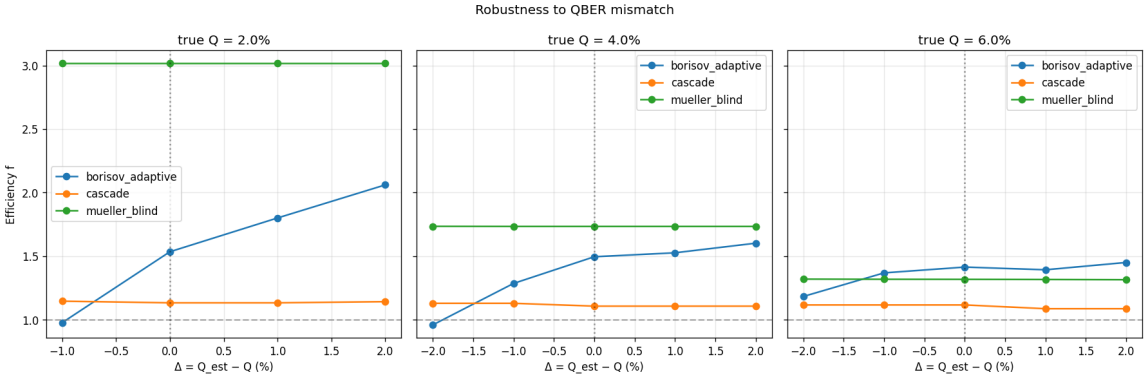
Definition

Robustness to QBER mismatch measures how much the reconciliation performance degrades when the estimated QBER $\hat{q}$ differs from the true QBER q. A robust protocol keeps f and the number of messages stable even under inaccurate estimates.

$$\Delta q = \hat{q} - q$$

- q - true QBER of the frame (known only after correction).
- $\hat{q}$ - estimated QBER used as input to the reconciliation algorithm.
- Δq - QBER mismatch; small $|\Delta q|$ keeps efficiency stable, while large $|\Delta q|$ inflates f and the number of messages.

Robustness to QBER mismatch ($N = 1024$)



Cascade is Δ-invariant; LDPC variants degrade when Q_{est} deviates from the true value.

Predicted secret-key rate

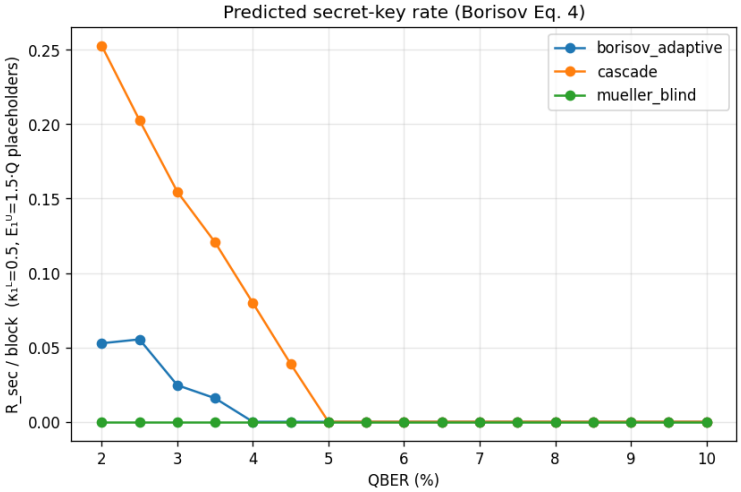
Definition

The **predicted secret-key rate** is the expected number of secret key bits produced per QKD round (or per second), after subtracting all information leaked during reconciliation, verification, and privacy amplification.

$$\ell_{\text{sec}} \simeq \ell_{\text{block}} \, (1 - \text{FER}) \, [\, 1 - h(q) - f \, h(q) \,]$$

- ℓ_{sec} - final secret-key length after privacy amplification.
- ℓ_{block} - raw key block length entering post-processing.
- FER - frame error rate (failed frames are discarded).
- $h(q)$ - binary entropy at QBER q; $1 - h(q)$ is the asymptotic secret fraction.
- $f \, h(q)$ - fraction of bits leaked during information reconciliation.
- f - reconciliation efficiency (closer to 1 $\Rightarrow$ more secret key).

Predicted secret-key rate ($N = 1024$)



R_{sec} per block under the Borisov Eq. 4 model with $\kappa_1^L = 0.5$, $E_1^U = 1.5Q$.

Future Work

Correctness fixes

- Regenerate the LDPC pool with per-rate Elkouss 2009 polynomials[a] (placeholder pool limits results at $R \geq 0.85$)
- Tighten BP iteration / disclosure retry logic in Borisov
- Sweep at $N \geq 4096$ with more frames per QBER point

[a] `https://ieeexplore.ieee.org/document/5205475`

Code optimization

- GPU-accelerate sparse $H \cdot x$ and BP message passing (CuPy[a])
- Batch-decode multiple frames per kernel launch

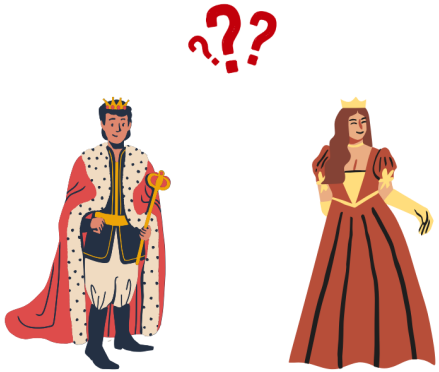
[a] `https://cupy.dev/`

Future Work (1)

Deployment

- Wire the harness into the faculty's QKD network (eventually) or a full SeQUeNCe[a] simulation after the two milestones above

[a] `https://github.com/sequence-toolbox/SeQUeNCe`



Conclusions

When to use what

- **Cascade** [Martinez-Mateo et al., 2015] - simple, robust to bad QBER estimates, but communication-heavy. Best when CPU is weak but the classical channel is fast.
- **LDPC blind** [Mueller et al., 2025] - fewer messages, proven on a live industrial QKD system. Best when bandwidth or latency on the classical channel matters.
- **Asymmetric LDPC** [Borisov et al., 2023] - almost as efficient as symmetric LDPC, but only one side does the heavy decoding. Best for thin transmitters (satellites, hand-helds, star networks).

References I

Berrou, C., Glavieux, A., and Thitimajshima, P. (1993).
Near Shannon limit error-correcting coding and decoding: Turbo-codes (1).
In *Proceedings of ICC '93 - IEEE International Conference on Communications*, volume 2, pages 1064–1070, Geneva, Switzerland. IEEE.

Borisov, N., Petrov, I., and Tayduganov, A. (2023).
Asymmetric adaptive ldpc-based information reconciliation for industrial quantum key distribution.
Entropy, 25(1):31.

Brassard, G. and Salvail, L. (1994).
Secret-key reconciliation by public discussion.
In *Advances in Cryptology — EUROCRYPT '93*, volume 765 of *Lecture Notes in Computer Science*, pages 410–423. Springer Berlin Heidelberg.

References II

Buttler, W. T., Lamoreaux, S. K., Torgerson, J. R., Nickel, G. H., Donahue, C. H., and Peterson, C. G. (2003).
Fast, efficient error reconciliation for quantum cryptography.
Physical Review A, 67(5):052303.

Elkouss, D., Leverrier, A., Alléaume, R., and Boutros, J. J. (2009).
Efficient reconciliation protocol for discrete-variable quantum key distribution.
In *IEEE International Symposium on Information Theory (ISIT)*, pages 1879–1883.

Gallager, R. G. (1962).
Low-density parity-check codes.
IRE Transactions on Information Theory, 8(1):21–28.

Jouguet, P. and Kunz-Jacques, S. (2014).
High performance error correction for quantum key distribution using polar codes.
Quantum Information and Computation, 14(3-4):329–338.

References III


Martinez-Mateo, J., Elkouss, D., and Martin, V. (2012).
Blind reconciliation.
Quantum Information and Computation, 12(9–10):791–812.


Martinez-Mateo, J., Pacher, C., Peev, M., Ciurana, A., and Martin, V. (2015).
Demystifying the information reconciliation protocol cascade.
Quantum Information and Computation, 15(5&6):453–477.
arXiv:1407.3257 [quant-ph].


Mueller, R., De Lazzari, C., Chirici, F., Vagniluca, I., Oxenløwe, L. K., Forchhammer, S., Zavatta, A., and Bacco, D. (2025).
Performance of cascade and ldpc codes for information reconciliation on industrial quantum key distribution systems.
IET Quantum Communication, 6(1):e70003.

References IV

Pacher, C., Grabenweger, P., Martinez-Mateo, J., and Martin, V. (2015).
An information reconciliation protocol for secret-key agreement with small leakage.
In *2015 IEEE International Symposium on Information Theory (ISIT)*, pages 730–734.

Wu, X., Kolar, A., Chung, J., Jin, D., Zhong, T., Kettimuthu, R., and Suchara, M. (2021).
Sequence: A customizable discrete-event simulator of quantum networks.
Quantum Science and Technology, 6(4):045027.